

Shown below is a sample response from the server:

```
<iq to="slyxuguy@gmail.com/Pidgin"
  id="7"
  type="result">
  <query xmlns="jabber:iq:roster">
    <item jid="user3@gmail.com/Home"/>
    <item jid="user4@gmail.com/Adium"/>
    <item jid="user4@gmail.com/Android"/>
  </query>
</iq>
```

You'll see multiple entries for *user4*—*user4@gmail.com/Adium* and *user4@gmail.com/Android*. *Adium* and *Android* are resource parameters, as discussed earlier. This tells us that *user4* has logged in from two clients.

Roster

A roster in XMPP is basically a buddy list, which contains a presence attribute for each user item. Your roster contains a list of Jabber user IDs (called JIDs) and the state of your presence subscriptions with those entities. When you come online, your client announces your presence to the server, and it handles the rest—both notifying your contacts that you are online, and fetching their current presence to display in your client interface. Your roster is updated when you send a presence stanza.

Here is a sample of an XML stream that combines each of the three types of stanzas in a session with the server. Note that this is not a debug output—the actual debug output streams will have many more details. This is just a structural example of an internal XML stream, intended to conceptually demonstrate interaction between client and server, and is taken from XMPP documentation. In this sample, the client (identified by “C:” in the sample) has authenticated with *gmail.com* as *user5@gmail.com*.

```
C: <stream:stream>
C: <presence/>
C: <iq type="get" id="1">
  <query xmlns="jabber:iq:roster"/>
</iq>
S: <iq type="result" id="1">
  <query xmlns="jabber:iq:roster">
    <item jid="user1@gmail.com"/>
    <item jid="user2@gmail.com"/>
    <item jid="user3@gmail.com"/>
  </query>
</iq>
C: <message from="user5@gmail.com"
  to="user2@gmail.com">
  <body>Hello world!</body>
</message>
S: <message from="user3@gmail.com"
  to="user5@gmail.com">
  <body>Kudos to you!</body>
```

```
</message>
C: <presence type="unavailable"/>
C: </stream:stream>
```

There can be many kinds of requests like the buddy add request, buddy remove request, group chat request, etc. All of these are handled by XML streams. XMPP includes a method to secure the stream from tampering and eavesdropping. This channel encryption method makes use of the Transport Layer Security (TLS) protocol, along with a “STARTTLS” extension that is modelled after similar extensions for the IMAP, POP3 and ACAP protocols.

Hands-on client code

We will now take a look at how we can write our own Google Talk XMPP client from scratch using Python, a simple yet powerful language that includes a staggering amount of functionality in its standard library—often referred to as ‘batteries included’. In addition, there are plenty of separately installable extensions and libraries for Python—for almost everything you can do with other languages.

For coding the client, we will use the XMPP module for Python, which you will probably need to install. If you are using a Debian-based distribution like Ubuntu, run the following command:

```
$ sudo apt-get install python-xmpp
```

For other distributions, if a package is not available in the repositories, download the tarball and install it as follows:

```
$ wget http://downloads.sourceforge.net/project/xmpppy/xmpppy/0.5.0-rc1/
xmppy-0.5.0rc1.tar.gz?use_mirror=ncch
$ tar -zxvf xmppy-0.5.0rc1.tar.gz
$ cd xmppy-0.5.0rc1
$ sudo python setup.py install
```

Now let us write the base client code. The code below will connect the client to the server and authenticate. A base client in just 12 lines of code—can you believe it?

```
#!/usr/bin/env python

import xmpp

user="username@gmail.com"
password="password"
server="gmail.com"

jid = xmpp.JID(user)
connection = xmpp.Client(server, debug[])
connection.connect()
result = connection.auth(jid.getNode(), password, "LYF-client")

connection.sendInitPresence()
```